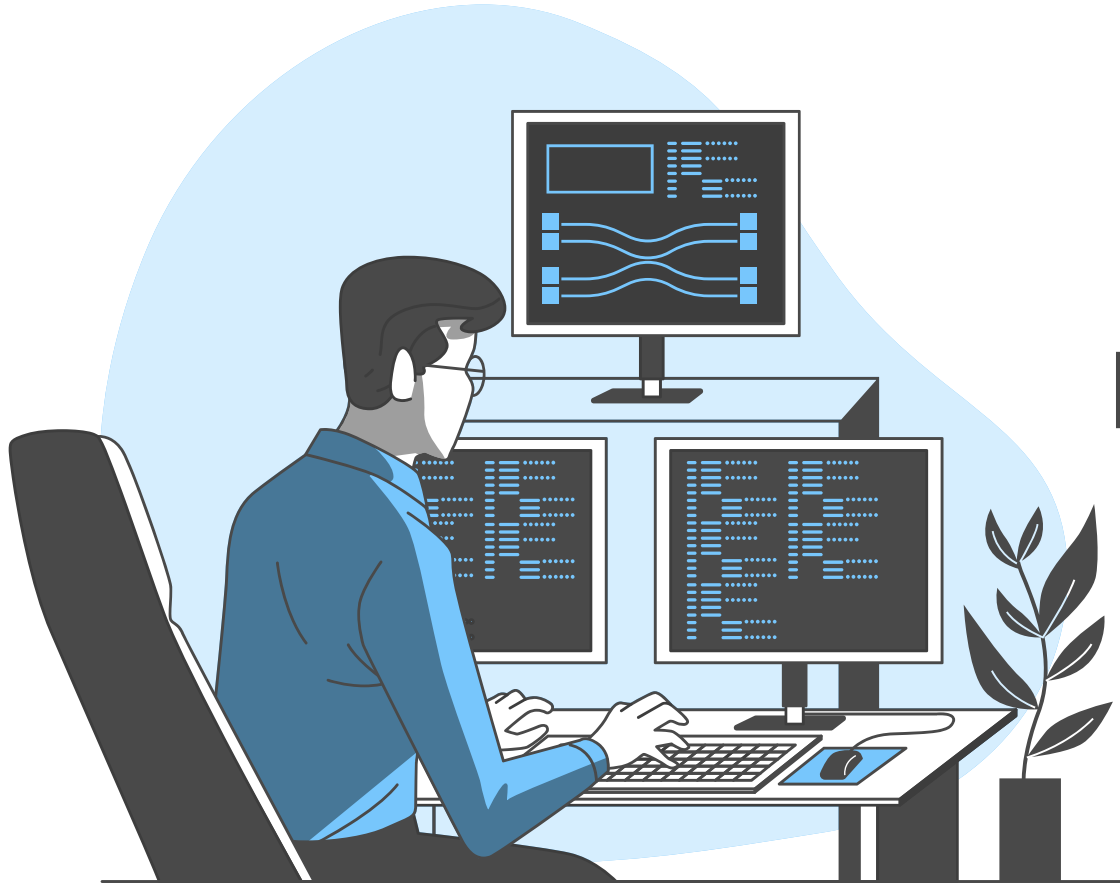




Sertifikasi Kompetensi Bidang TIK

Database Programmer

Pretest

- Apa itu Database?
- Bagaimana cara memodelkan data fisik menjadi database?
- Bagaimana cara memanipulasi data yang ada di dalam database?

Database

- Data

- Keterangan yang benar dan nyata
- Keterangan atau bahan nyata yang dapat dijadikan dasar kajian (analisis atau kesimpulan)

Database

- Base / Basis

- Pangkalan (angkatan laut, angkatan darat, dan sebagainya) untuk melakukan operasi

Database

kumpulan data yang disimpan secara terstruktur di dalam komputer dan diatur sehingga mudah dalam pengelolaannya.

Database



>>



Database

- Keuntungan

- Terkontrolnya data
- Terpeliharanya ke-konsistenan data
- Data dapat dipakai secara bersama (shared)
- Dapat diterapkan standarisasi
- Keamanan data terjamin
- Terpeliharanya integritas data
- Data independence (kemandirian data)

Database

- Kelemahan

- rumit, sulit, dan memakan waktu untuk dirancang
- Biaya awal besar
- Kerusakan database mempengaruhi hampir semua program aplikasi
- Biaya konversi yang ekstensif
- Pelatihan awal diperlukan untuk semua programmer dan pengguna

Karakteristik Database

- **Atomicity**

menjamin bahwa setiap transaksi diperlakukan sebagai satu "unit", yang berhasil sepenuhnya atau gagal sepenuhnya: jika salah satu pernyataan yang merupakan transaksi gagal diselesaikan, seluruh transaksi gagal dan database dibiarkan tidak berubah.

- **Isolation**

memastikan bahwa eksekusi transaksi secara bersamaan meninggalkan database dalam keadaan yang sama yang akan diperoleh jika transaksi dieksekusi secara berurutan.

- **Consistency**

memastikan bahwa transaksi hanya dapat membawa database dari satu status valid ke status valid lainnya, mempertahankan invarian database: data apa pun yang ditulis ke database harus valid menurut semua aturan yang ditentukan, termasuk batasan, kaskade, pemicu, dan kombinasinya.

- **Durability**

menjamin bahwa sekali transaksi telah dilakukan, itu akan tetap dilakukan bahkan dalam kasus kegagalan sistem

Entity Relationship Diagram

- Entity dan Relationship atau dikenal dengan Entity Relationship Diagram merupakan langkah awal dalam membangun sebuah database.
- Fungsinya adalah menunjukkan relasi hubungan (Relationship) antara objek (Entity) dalam sistem database.

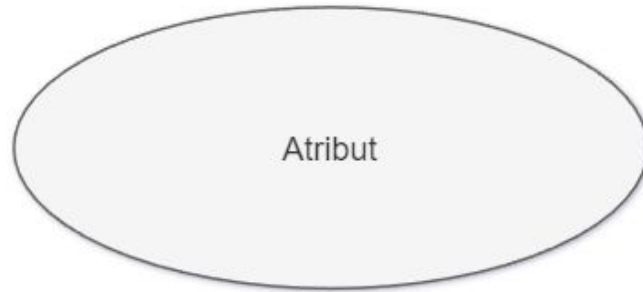
Komponen ERD

- Entity/Entitas
 - Entitas atau Entity mewakili objek yang unik. Entity disimbolkan dengan persegi panjang.
 - Contoh entitas di toko sembako: barang.



Komponen ERD

- Atribut
 - Atribut merupakan deskripsi atau karakteristik dari sebuah Entity.
 - Simbol dari Atribut menggunakan elips.



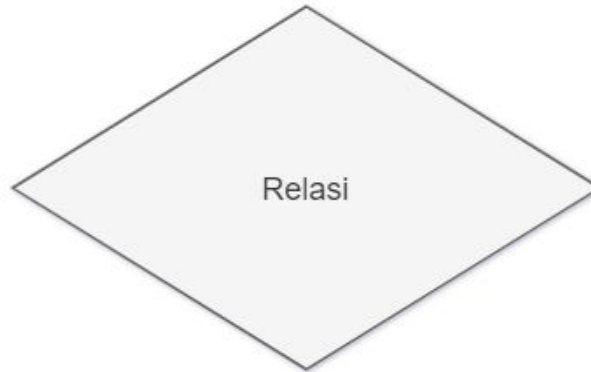
Komponen ERD

- Atribut
 - Jenis atribut
 - Atribut Kunci.
 - Atribut simple
 - Atribut multi nilai
 - Atribut gabungan
 - Atribut derivatif

Komponen ERD

- ## Relation

- Relasi atau Relation merupakan hubungan dari antara Entity yang berbeda dan biasanya
- Disimbolkan dengan belah ketupat.



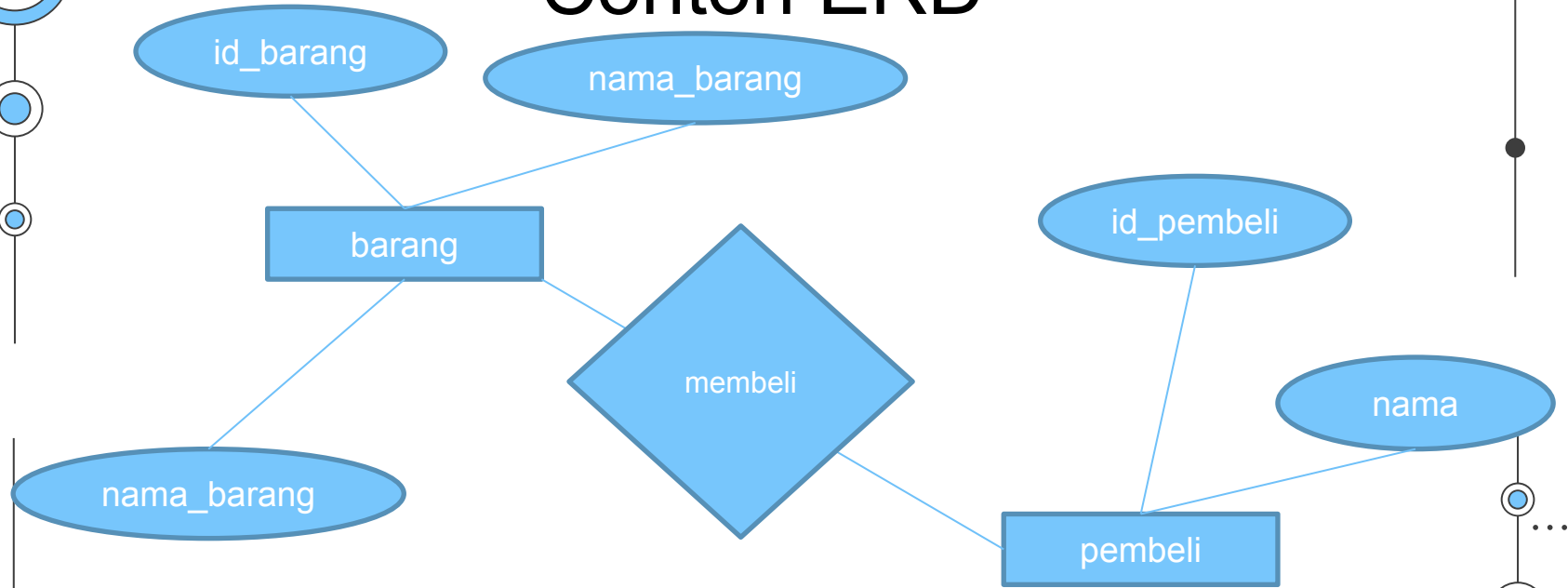
Komponen ERD

- Relation
 - Jenis Relation
 - One to One
 - One to Many
 - Many to Many
 - Relation nantinya dapat menjadi table tersendiri.

Komponen ERD

- Penghubung
 - Berbentuk garis yang menghubungkan entitas

Contoh ERD



Normalisasi

- Normalisasi merupakan teknik logical desain yang mengelompokkan atribut untuk membentuk entitas sederhana, fleksibel, *nonredundant*, dan mudah beradaptasi.

Normalisasi

- Contoh table tidak normal

id_nota	id_pembeli	nama_pembeli	kode_barang	nama_barang	jumlah	Harga
1	p1	agus	b1	gula	2	5000
			b2	kopi	2	2000
2	p2	budi	b1	gula	3	5000
3	p3	bobby	b2	kopi	5	2000

Normalisasi

- Langkah-langkah
 - 1NF
 - Suatu tabel dikatakan 1NF ketika setiap atribut dari data tersebut hanya memiliki nilai tunggal dalam satu baris.

Normalisasi

- Contoh bentuk 1NF

id_nota	id_pembeli	nama_pembeli	kode_barang	nama_barang	jumlah	Harga
1	p1	agus	b1	gula	2	5000
1	p1	agus	b2	kopi	2	2000
2	p2	budi	b1	gula	3	5000
3	p3	bobby	b2	kopi	5	2000

Normalisasi

- Langkah-Langkah
 - 2NF
 - Suatu table dikatakan 2NF Ketika *sudah tidak ditemukan partial “functional dependency” kepada primary key dalam sebuah tabel.*
 - **functional dependency** adalah Ketika setiap atribut yang non key bergantung secara fungsional terhadap primary key.

Normalisasi

- Contoh bentuk 2NF

id_nota	id_pembeli	jumlah	kode_barang
1	p1	2	b1
1	p1	2	b2
2	p2	3	b1
3	p3	5	b2

id_pembeli	nama_pembeli
p1	agus
p2	budi
p3	bobby

kode_barang	nama_barang	Harga
b1	gula	5000
b2	kopi	2000

Normalisasi

- Langkah-Langkah
 - 3NF
 - Suatu table dikatakan 3NF Ketika tidak diperkenankan adanya partial “**transitive dependency**” dalam sebuah tabel.

Normalisasi

- Contoh bentuk 3NF

id_nota	id_pembeli
1	p1
2	p2
3	p3

id_pembeli	nama_pembeli
p1	agus
p2	budi
p3	bobby

id_nota	kode_barang	jumlah
1	b1	2
1	b2	2
2	b1	3
3	b2	5

kode_barang	nama_barang	Harga
b1	gula	5000
b2	kopi	2000

Latihan 1

no	nama mobil	Jenis mobil	harga sewa	Kapasitas penumpang	Tahun pembuatan	fasilitas
1	APV	MPV	300000	9	2005	AC, Sound System, GPS
2	Atoz	City Car	300000	4	2007	AC, Sound System
3	Avanza	MPV	250000	8	2006	AC, Sound System, GPS, TV
4	Aveo	City Car	250000	4	2004	AC, Sound System, GPS, heater

Struktur Data

- Database dapat menyimpan tipe data yang berbeda-beda pada setiap kolom table.
- Saat membuat tabel diharuskan mendefinisikan tipe data untuk setiap fieldnya.
- Secara umum tipe data yang digunakan dibagi 3 macam yaitu string, numerik, dan date and time.

Struktur Data – String

Tipe Data	keterangan
CHAR	Mampu menyimpan data hingga 255 karakter.
VARCHAR	Mampu menyimpan data hingga 255 karakter.
FTINYBLOB, TINYTEXT	Mampu menyimpan data hingga 2^8-1 (255) data.
BLOB, TEXT	Mampu menyimpan data hingga $2^{16}-1$ (16M-1) data.
MEDIUMBLOB, MEDIUMTEXT	Mampu menyimpan data hingga $2^{24}-1$ data
LOB, LONGTEXT	Mampu menyimpan data hingga berukuran GIGA BYTE.
ENUM('value 1', 'value 2',, 'valueN')	Berukuran 1 atau 2 byte tergantung jumlah nilai enum (maksimum 65535 nilai)
SET('value 1', 'value 2', 'Value N')	Berukuran 1, 2, 3, 4 atau 8 byte tergantung jumlah anggota himpunan (Maksimal 64 anggota)

Struktur Data – Numerik

Tipe Data	Keterangan
TINYINT	Berukuran 1 byte. Bilangan Bulat. Bilangan bertanda: -128 ~ 127. Bilangan tidak bertanda: 0 ~ 255.
SMALLINT	Berukuran 2 Byte. Bilangan Bulat. Bilangan bertanda: -32768 ~ 32767. Bilangan tidak bertanda : 0 ~ 65535.
MEDIUMINT	Berukuran 3 Byte. Bilangan Bulat. Bilangan bertanda : -8388608 ~ 8388607. Bilangan tidak bertanda : 0 ~ 16777215.
INT	Berukuran 4 byte. Bilangan Bulat. Bilangan bertanda : -2147483648 ~ 2147483647. Bilangan tidak bertanda : 0 ~ 4294967295.
BIGINT	Berkuran 8 byte. Bilangan Bulat. Bilangan bertanda : -9223372036854775808 ~ 9223372036854775807. Bilangan tidak bertanda : 0 ~ 1844674473709551615.
FLOAT	Berukuran 4 byte. Bilangan pecahan.
DOUBLE	Berukuran 8 byte. Bilangan pecahan.
DECIMAL(M, D)	Berukuran M byte. Bilangan pecahan. Contoh: (5,2) dapat menyimpan bilangan -99,99 sampai dengan 99,99



Struktur Data - Date

Tipe Data	Keterangan
DATETIME	Berukuran 8 byte. Kombinasi tanggal dan jam. Jangkauan: '1000-01-01 00:00:00' ~ '9999-12-31 23:59:59'
DATE	Berukuran 3 byte. Tanggal Jangkauan dari '1000-01-01' ~ '9999-12-31'
TIMESTAMP	Berukuran 4 byte.
TIME	Berukuran 3 byte.
YEAR	Berukuran 1 byte. Data tahun antara 1901 ~ 2155

SQL

- Structured Query Language merupakan bahasa standar dalam mengakses dan memanipulasi database.
- Perintah atau command pada SQL dibagi menjadi beberapa tipe, berikut tipe - tipe yang ada pada bahasa SQL.
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
 - Data Query Language
 - Data Control Language
 - Transaction Control Language

SQL-DDL

Perintah	Keterangan
CREATE	Membuat objek seperti tabel, view, procedure, dan lain-lain
ALTER	Mengubah struktur dan definisi objek yang ada
RENAME	Mengganti nama objek
DROP	Menghapus database dan tabel
TRUNCATE	Menghapus semua record dalam tabel database
COMMENT	Menambahkan dokumentasi komentar di objek

SQL-DDL

- Contoh
 - CREATE DATABASE warung;
 - CREATE TABLE barang (
kode_barang INT PRIMARY KEY,
nama_barang VARCHAR(255) NOT NULL,
harga VARCHAR(255) NOT NULL,
stok VARCHAR(255) NOT NULL
);
 - ALTER TABLE barang CHANGE harga
harga INT(11) NOT NULL;
 - ALTER TABLE barang DROP COLUMN
stok;

SQL-DML

Perintah	Keterangan
SELECT	Menampilkan data dari satu atau lebih tabel
INSERT	Menambahkan data baru di database
UPDATE	Mengubah data yang ada di database
DELETE	Menghapus data yang ada di database

SQL-DQL

Klausa/Operator	Keterangan
WHERE	Filter pencarian data
AND, OR, NOT	Operator pelengkap WHERE
LIKE	Operator pelengkap WHERE
ORDER BY	Mengurutkan data secara A-Z / Z-A
LIMIT	Membatasi hasil pencarian
AGGREGATION	Fungsi agregasi
GROUP BY	Mengelompokkan nilai berdasarkan field

SQL-DML

- Contoh
 - `INSERT INTO barang (id, nama_barang, harga) VALUES ('1', 'gula', '3000')`
 - `UPDATE barang SET nama_barang = 'gulaku' WHERE barang.id = 1;`
 - `DELETE FROM detail_pembelian WHERE `detail_pembelian`.`id` = 1`
 - `SELECT` `* FROM `barang`

SQL-DCL

Perintah	Keterangan
GRANT	Memberikan hak akses pengguna ke database
REVOKE	Menghilangkan hak akses yang telah diberikan dengan perintah GRANT

SQL-TCL

Perintah	Keterangan
COMMIT	Digunakan untuk melakukan transaksi
ROLLBACK	Digunakan untuk membatalkan transaksi

Stored Procedure

- Stored Procedure merupakan suatu blok program yang dapat dipanggil berulang-ulang.
- Contoh
 - CREATE PROCEDURE
kurang_satu(number int) RETURNS int(11)
BEGIN
return number - 1;
END
 - Call kurang_satu(3)

Stored Function

- Mirip Stored procedure namun stored function harus mengembalikan nilai sedangkan stored procedure tidak harus mengembalikan nilai.
- Contoh
 - ```
CREATE FUNCTION
 harga_transaksi(id_barang INT(11),
 jumlah INT(11))
 RETURNS INT(11)
 BEGIN
 SELECT harga into @harga FROM barang
 WHERE id = id_barang;
 return @harga * jumlah;
 END
```
  - ```
Select harga_transaksi(1,5)
```

Trigger

- Trigger merupakan blok kode yang otomatis akan tereksekusi secara otomatis sesuai kondisi yang telah ditentukan.
- Contoh
 - CREATE TRIGGER
kurang_stok_barang_saat_transaksi
BEFORE INSERT ON detail_transaksi
FOR EACH ROW
BEGIN
UPDATE barang SET stok = stok - new.jumlah
WHERE id = new.barang_id;
END

TRANSACTION

- Perintah dalam SQL yang berfungsi membuat semua perubahan yang ditunda menjadi permanent dengan perintah commit.
- Sedangkan untuk Rollback berfungsi untuk membatalkan semua perubahan yang ditunda.

Latihan 2

- Buatlah database berdasarkan dari table yang sudah di normalisasi pada Latihan 1
- Tampilkan nama mobil, jenis mobil, fasilitas mobil
- s.itk.ac.id/database2022hari1

Database Access

- Mengontrol data yang dapat diakses oleh pengguna merupakan suatu hal yang penting dilakukan.
- Hal ini dapat dilakukan memberikan akses kontrol pada DBMS.
- Membuat user
 - `CREATE USER 'database_user'@'localhost' IDENTIFIED BY 'user_password';`
- Mengubah password user
 - `ALTER USER 'database_user'@'localhost' IDENTIFIED BY 'new_password';`

Database Access

- menghapus User
 - DROP USER
'database_user@'localhost';
- memberikan hak akses
 - GRANT SELECT, INSERT, DELETE ON
database_name.* TO
database_user@'localhost';
- Menampilkan semua user
 - SELECT User, Host, Password FROM
mysql.user;

Database Access

- menghapus akses
 - `REVOKE ALL PRIVILEGES ON database_name.* TO 'database_user'@'localhost';`
- memeriksa akses
 - `SHOW GRANTS FOR 'database_user'@'localhost'`

Database Optimization

- Trigger
 - 1 operasi bisa mempengaruhi table lain
 - Otomasi
- Store procedure
 - Mengurangi boiler plate
 - +access = keamanan++
- Function
 - Mengurangi boiler plate
- View
 - Meringankan kinerja dengan table virtual
 - Mengurangi boiler plate
- Transaction
 - Memastikan semua query sukses
 - Konsistensi data terjaga
- Index tanya sama anandi
 - Mempercepat query pencarian



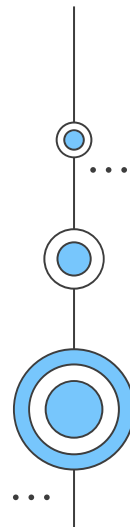
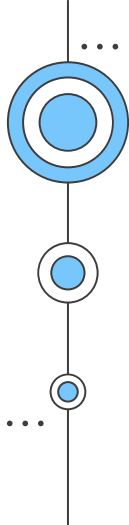
Gunakan
dengan
bijak

Database Optimization

- Trigger
 - Otomasi
- SP
 - Reusable
 - +access == aman
 - Precompile >> faster
- View
 - Lebih cepat
 - Reusable
 - Simple
 - Secure
- Function
 - Reusable
 - Simplify
- Index
 - Faster
- transaction



Gunakan
dengan
bijak



Latihan 3

- Buatlah pengguna dengan nama user_dbwarung!
- Berikan hak akses untuk melakukan select pada database warung!

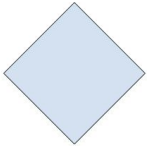
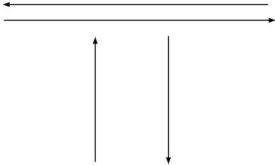
Programming Algorithm

- **Algorithm** atau algoritma langkah-langkah yang disusun berurutan yang digunakan untuk menyelesaikan suatu permasalahan.
- **Programming Algorithm** adalah suatu prosedur atau formula yang berisi langkah-langkah perintah yang akan dilakukan komputer untuk menyelesaikan suatu masalah.

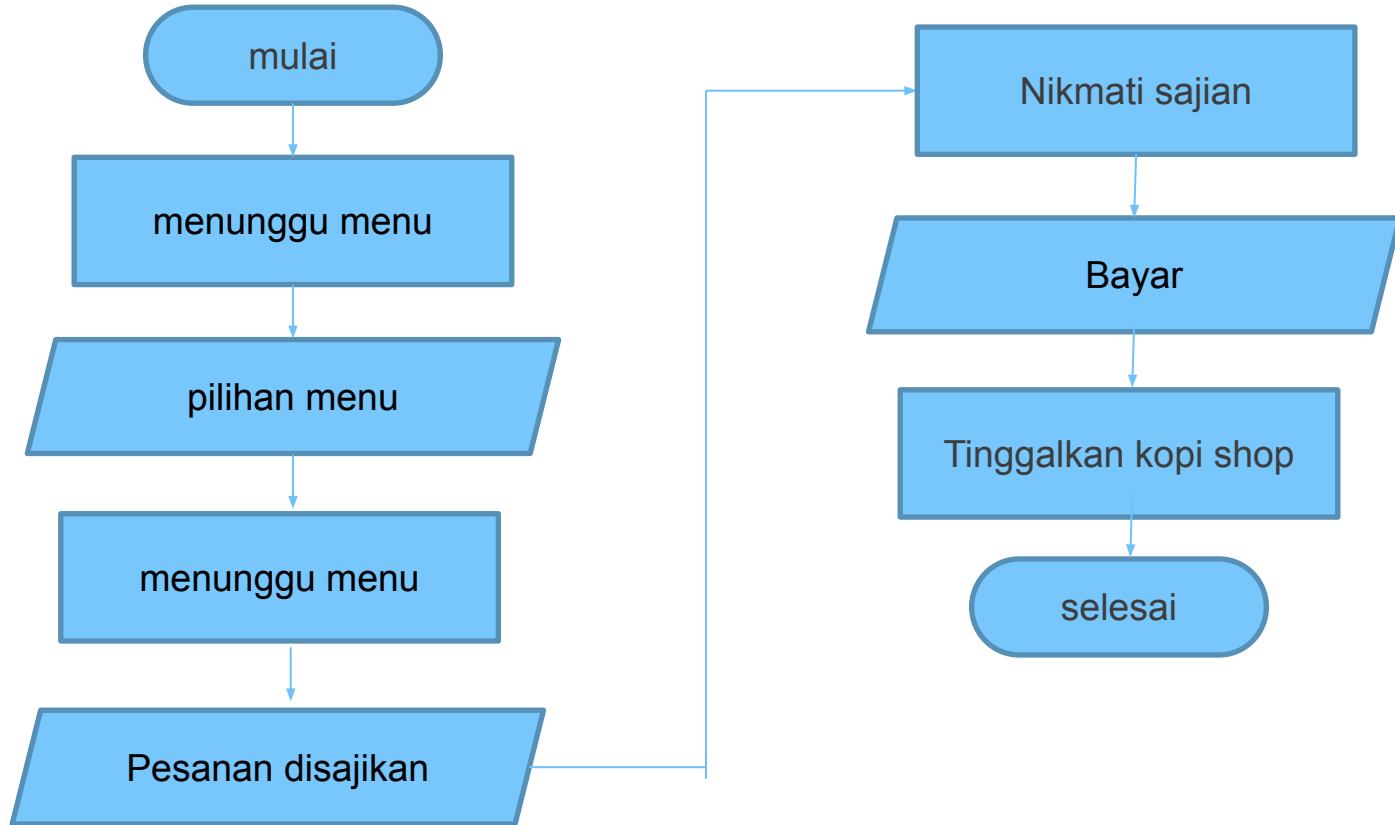
Programming Algorithm

- Contoh Algoritma pemesanan di suatu kedai kopi
 1. Masuk kedalam kedai kopi
 2. Tentukan pilihan menu
 3. Tunggu menu yang dipesan disajikan
 4. Menu yang anda pesan disajikan
 5. Nikmati sajiannya
 6. Lakukan pembayaran
 7. Keluar dari kedai kopi

Programming Algorithm

Simbol	Nama	Fungsi
	Terminator	Melambangkan awal dan akhir sebuah flowchart
	Input - Output	Melambangkan proses dari sebuah input dan output
	Proses	Melambangkan proses pengelolaan yang dilakukan program
	Decision	Melambangkan pemilihan kondisi
	Flow Direction Symbol	Melambangkan hubungan dan relasi dengan simbol simbol lain

Programming Algorithm



Programming Algorithm

- **Psedocode** merupakan bentuk penulisan algoritma yang paling mendekati penulisan code.
- **INPUT**. Digunakan untuk menunjukan proses memasukkan nilai kedalam variable
- **OUTPUT**. Digunakan untuk menunjukkan proses keluaran yang telah terjadi
- **WHILE**. Digunakan untuk melakukan sebuah iterasi perulangan yang memiliki iterasi awal
- **FOR**. Digunakan untuk sebuah perhitungan iterasi
- **REPEAT – UNTIL**. Digunakan untuk sebuah perulangan iterasi yang memiliki kondisi akhir
- **IF - THEN – ELSE**. Digunakan untuk mengambil sebuah keputusan dari beberapa buah kondisi

Programming Algorithm

- Contoh Pseudocode
Input: menu
Menunggu menu
Output: pesanan
Nikmati pesanan
Input: bayar

Programming Algorithm

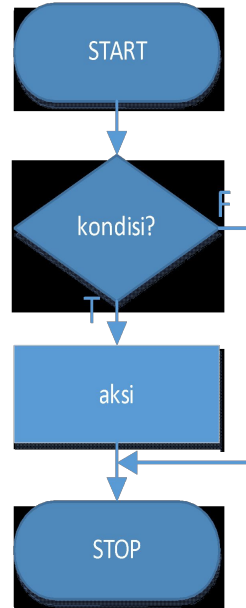
- Percabangan
 - yaitu instruksi yang dikerjakan dengan kondisi tertentu. Kondisi adalah persyaratan yang dapat bernilai benar atau salah. Satu atau beberapa instruksi hanya dilaksanakan apabila kondisi bernilai benar, sebaliknya apabila salah maka instruksi tidak akan dilaksanakan.

Programming Algorithm

- Percabangan
 - Percabangan untuk satu kondisi (if)
 - Percabangan untuk satu kondisi dengan aksi ketika salah (if else)
 - Percabangan untuk lebih dari satu kondisi (if else bertingkat)

Programming Algorithm

- Percabangan

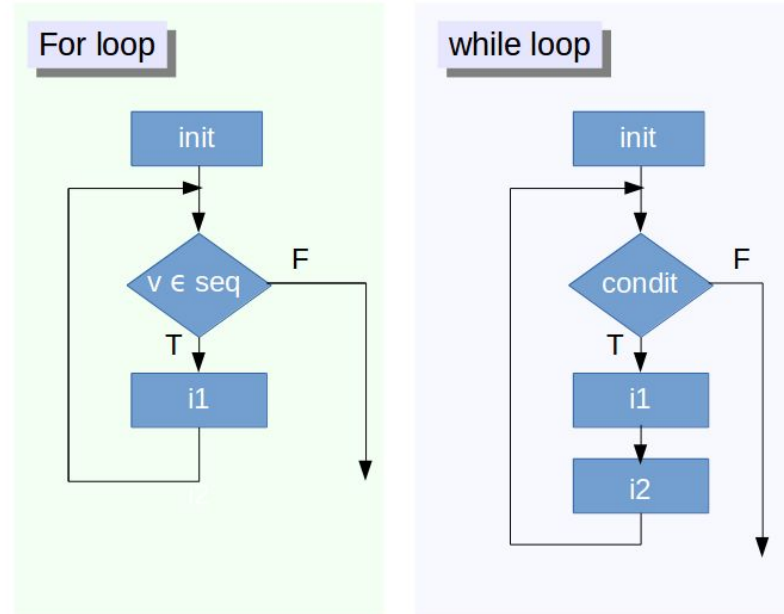


Programming Algorithm

- Perulangan
 - Yaitu instruksi khusus dalam bahasa pemrograman untuk mengulang beberapa perintah sesuai dengan instruksi yang ditentukan. Bertujuan untuk mempermudah pengerjaan program dan untuk mempersingkat instruksi program.

Programming Algorithm

- Perulangan

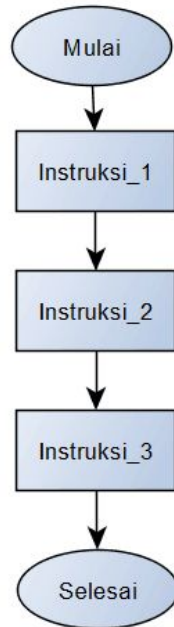


Programming Algorithm

- Sekuensial (Runtunan)
 - Yaitu algoritma yang bekerja dengan cara mengeksekusi setiap instruksi secara berurutan. Setiap instruksi dikerjakan satu per satu pada setiap barisnya dari awal hingga akhir.

Programming Algorithm

- Sekuensial



Algoritma Sorting

- Bubble Sorting
 - Algoritma pengurutan yang membandingkan masing-masing node dalam suatu list secara berpasangan, menukar node jika diperlukan, dan mengulanginya sampai akhir dari list telah berurut.

Algoritma Sorting

- Bubble Sort

Bubble Sort



Algoritma Sorting

- Insertion Sorting
 - Algoritma yang memilah data yang akan diurutkan menjadi dua bagian. Elemen pertama diambil dari bagian array yang belum diurutkan dan kemudian diletakkan sesuai posisinya pada bagian lain dari array yang telah diurutkan

Algoritma Sorting

- Insertion Sort

Insertion Sort



Algoritma Sorting

- Selection Sorting
 - Algoritma yang memilah data dengan nilai paling rendah dan menukar elemen yang terpilih dengan elemen ke i . Nilai dari i dimulai dari ke n , dimana n adalah jumlah total elemen dikurang 1.

Algoritma Sorting

- Selection Sort



Algoritma Searching

- Linear Search
 - Metode algoritma yang mencari secara berurutan memeriksa setiap elemen daftar sampai kecocokan ditemukan atau seluruh daftar telah dicari.

Algoritma Searching

- Linear Search

5	4	3	2	1
---	---	---	---	---

searching for 3

5	4	3	2	1
---	---	---	---	---

5 == 3? No, next!

5	4	3	2	1
---	---	---	---	---

4 == 3? No, next!

5	4	3	2	1
---	---	---	---	---

3 == 3? Yes, found it!

Algoritma Searching

- Binary Search
 - Algoritma yang melakukan pencarian dengan membandingkan nilai target dengan elemen tengah array

Algoritma Searching

- Binary Search

11	15	21	31	33
----	----	----	----	----

Cari 11

11	15	21	31	33
----	----	----	----	----

Cari ke Kiri

11	15	21	31	33
----	----	----	----	----

Ketemu!

Debugging

- **Debugging** merupakan proses meminimalkan kecacatan yang ada dalam perangkat lunak yang dikembangkan.
- **Debugging** dapat dilakukan dengan melakukan pengujian terhadap perangkat lunak kemudian memperbaikinya.
- **Hal yang diperlukan** saat melakukan pengujian adalah dibutuhkannya test case.

<Input, output yang diharapkan>
<stok barang = 20, tidak ada barang>

Debugging

- **Teknik-Teknik penyusunan test case**
 - Error Guessing
 - Euqivalence Partitioning
 - Boundary Value Analysis
 - Decision Table Technique

Setelah seluruh proses pengujian selesai dilakukan lalu pengembang dapat melakukan Perbaikan. Proses ini berulang hingga kecacatan dapat dibuat seminimal mungkin.

Debugging

- **Error Guessing**
 - Tester akan mencoba berbagai macam scenario yang akan menghasilkan error.
 - Bergantung pada imajinasi dan kemampuan berpikir analitik dari tester
 - Setiap tester akan menggunakan scenario berbeda

Debugging

- **Equivalence Partitioning**
 - Digunakan untuk mendesain test dengan inputan angka dengan range besar.
 - Dilakukan dengan membagi inputan menjadi beberapa bagian dan menggunakan salah satu nilai di dalam range sebagai test input.

Debugging

- **Boundary Value Analysis**
 - Digunakan untuk mendesain test dengan inputan angka dengan range $a - z$
 - Maka test yang digunakan $a - 1, a, a + 1, z - 1, z, z + 1$.

Debugging

- **Decision Table Technique**
 - Perancangan test case dengan menggunakan tabel kebenaran
 - Dilakukan dengan mendaftar seluruh kemungkinan scenario yang mungkin terjadi.

Code Review

Source code dapat diakses melalui

s.itk.ac.id/database_hari2

Atau

<https://github.com/Uskrai/pelatihan-tik-database-2022>

Code Review

Source code dapat diakses melalui

s.itk.ac.id/database_hari2

Atau

<https://github.com/Uskrai/pelatihan-tik-database-2022>

Ujian

- Temukan suatu kasus yang membutuhkan database dan buat ERD dan normalisasi nya!
- Buatlah database berdasarkan rancangan yang sudah dibuat!
- Buatlah API untuk database yang sudah dibuat!
- Manfaatkanlah Stored procedure, stored function, trigger dan transaksi dalam aplikasi api yang dibuat!

